

Cruscotto Degenze

Documento di progetto: architettura, modello dati e piano di realizzazione

Redatto da Mihnea Sever Molnar · Versione 1.0 · 14 settembre 2026

1. Obiettivo

Sostituire la gestione informale dell'agenda di reparto con un'applicazione web interna, consultabile e aggiornabile da più postazioni, che dia a medici e infermieri la situazione dei pazienti ricoverati giorno per giorno.

L'agenda è organizzata su tre reparti: AOUP, dove la degenza arriva a dieci giorni, e i due Setting di Low Care, dove arriva a venti. Per ogni paziente si registrano i dati identificativi, il posto letto, un indicatore di gravità e una nota per ciascun giorno di degenza, più due sezioni di note estese separate, una medica e una infermieristica.

Cosa il programma fa

- Autenticazione con utenze assegnate dall'amministratore e cinque livelli di permesso.
- Visualizzazione dei tre reparti in un'unica pagina, con la griglia dei giorni di degenza.
- Inserimento, modifica ed eliminazione dei pazienti.
- Note giornaliere e note cliniche separate per categoria professionale.
- Esportazione dei dati in CSV per l'amministrazione.
- Tracciamento di chi ha modificato cosa e quando.

Cosa il programma non fa

- Non è una cartella clinica e non sostituisce il gestionale sanitario: è uno strumento organizzativo di reparto.
- Non si integra con altri applicativi della struttura.
- Non gestisce prescrizioni, terapie, referti o immagini.

2. Scelte tecnologiche

- **Python e Flask.** È il linguaggio che già conosco. Flask con i template Jinja2 genera pagine HTML lato server, che per un'applicazione tabellare come questa è molto più rapido da scrivere di un'interfaccia a componenti.
- **SQLAlchemy.** È la scelta che rende indolore il passaggio da SQLite a PostgreSQL: cambia la stringa di connessione, non la logica.
- **SQLite in locale, PostgreSQL in rete.** SQLite è un singolo file, non richiede installazione né servizi e si copia con un comando: perfetto per la fase di prova. Non regge però scritture concorrenti da più postazioni, che è esattamente lo scenario della fase successiva.
- **Nessun framework frontend.** HTML, CSS e un po' di JavaScript bastano per questa interfaccia e non introducono una catena di compilazione da mantenere.

3. I due ambienti

Ambiente locale

L'applicazione gira su un singolo PC e resta in ascolto sull'indirizzo di loopback `127.0.0.1`. Solo quel computer può raggiungerla: nessun altro dispositivo della rete la vede, il firewall di Windows non chiede autorizzazioni e non serve aprire alcuna porta. Il database è il file `degenze.db` nella cartella del progetto, e il backup consiste nel copiarlo.

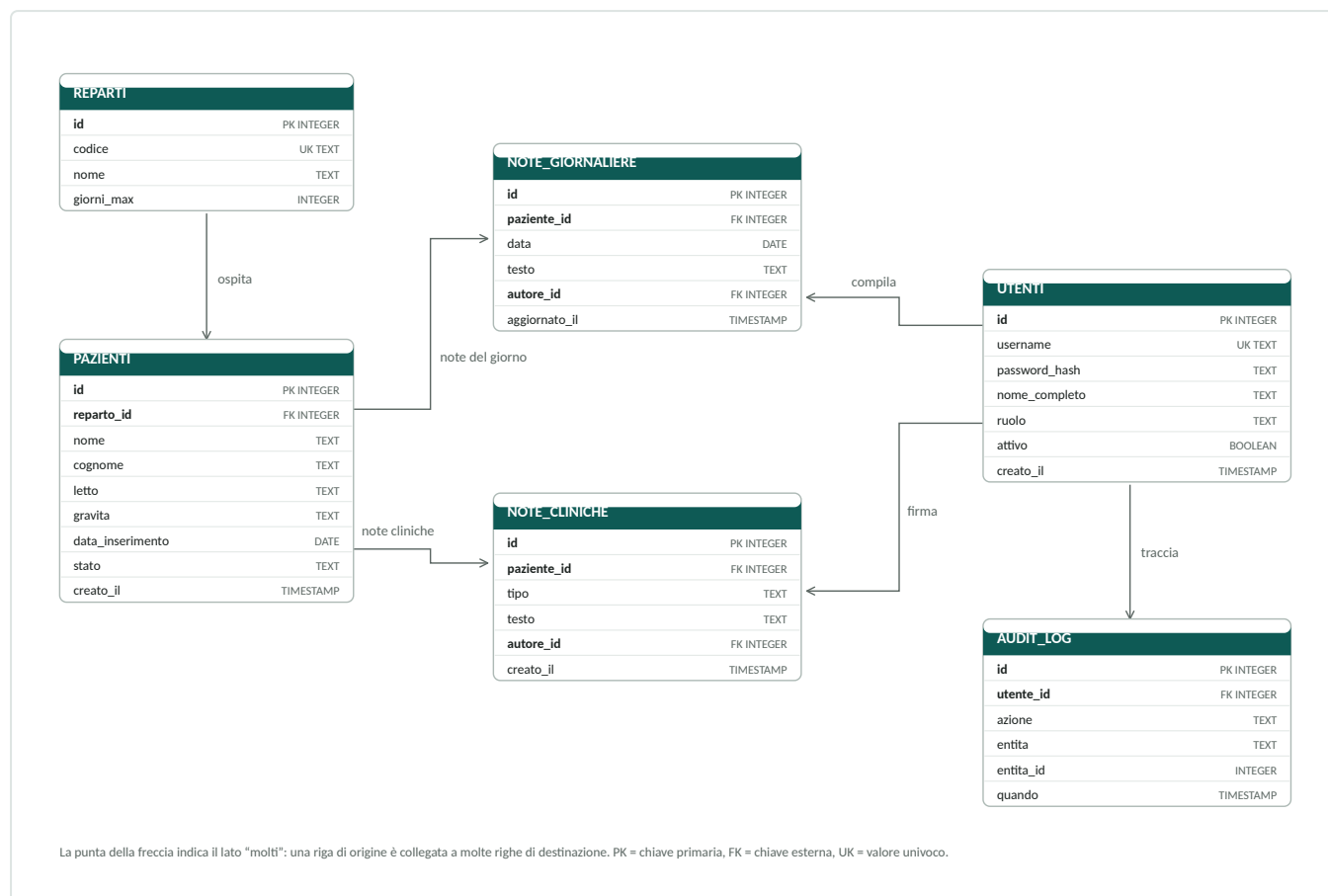
Ambiente di rete

L'applicazione viene installata su una macchina virtuale del datacenter. Gunicorn esegue il codice Python e resta in ascolto solo su `127.0.0.1:8000`, mentre Nginx riceve le richieste sulla porta 80 e le inoltra a Gunicorn. Questo doppio livello serve: Gunicorn esegue l'applicazione, Nginx gestisce le connessioni dei client, i file statici e, in futuro, il certificato per HTTPS. I dati stanno in PostgreSQL sulla stessa macchina.

4. Modello dati

La scelta strutturale su cui poggia tutto il resto è che **le note giornaliere sono righe, non colonne**. La tentazione naturale sarebbe creare una tabella pazienti con i campi giorno1, giorno2 e così via fino a giorno10 o giorno20. Non funziona per tre motivi: i reparti hanno durate diverse e servirebbero due strutture distinte; cambiare il numero di giorni richiederebbe di modificare la struttura del database in produzione; e non si saprebbe chi ha scritto una certa nota né quando.

Con una riga per coppia paziente-data, la griglia a dieci o venti colonne viene ricostruita al momento della visualizzazione: si parte dalla data di inserimento del paziente, si generano le date successive fino al limite del reparto, e per ognuna si cerca la riga corrispondente. Se non esiste, la cella è vuota. Il limite di giorni diventa così un campo della tabella reparti, modificabile senza toccare il codice.



Schema entità-relazioni del Cruscotto Degenze.

Le tabelle

- **utenti** — Le utenze applicative. Il campo **ruolo** determina i permessi; **attivo** consente di disabilitare un'utenza senza cancellarla, cosa necessaria perché le note restano collegate al loro autore.
- **reparti** — Tre righe: AOUP, Setting 1 e Setting 2. Il campo **giorni_max** vale 10 per il primo e 20 per gli altri due.
- **pazienti** — Anagrafica minima più posto letto, gravità e data di inserimento. Da quest'ultima si calcolano tutte le colonne della griglia.
- **note_giornaliere** — Il contenuto delle celle. Il vincolo di unicità sulla coppia paziente e data impedisce che si creino due note per lo stesso giorno.

- **note_cliniche** — Le note estese, distinte dal campo `tipo` con valore MEDICO o INFERMIERE. Sono voci in sola aggiunta, ciascuna con il suo autore e la sua data: una nota nuova non cancella le precedenti, come avviene in un diario di reparto.
- **audit_log** — Registra le operazioni. In ambito sanitario poter ricostruire chi ha modificato cosa non è un di più, ed è molto più fastidioso aggiungerlo dopo che prevederlo dall'inizio.

5. Ruoli e permessi

- **Amministratore** — Lettura e scrittura su tutto, creazione e disattivazione delle utenze, eliminazione dei pazienti.
- **Medico** — Legge tutto. Scrive le note giornaliere e le note cliniche di tipo medico. Vede le note infermieristiche ma non le modifica.
- **Infermiere** — Legge tutto. Scrive le note giornaliere e le note cliniche di tipo infermieristico. Vede le note mediche ma non le modifica.
- **OSS** — Sola lettura.
- **Amministrazione e IT** — Sola lettura, più l'esportazione dei dati in CSV.

Il controllo va fatto sul server a ogni richiesta. Nascondere un pulsante nella pagina è utile all'utente ma non è una misura di sicurezza: chi conosce l'indirizzo della funzione potrebbe richiamarla direttamente.

Password

L'impostazione scelta è deliberatamente minimale: le utenze le crea l'amministratore e le password le comunica di persona. Non c'è registrazione autonoma, non c'è recupero password, non ci sono requisiti di complessità né scadenze. Nel database però le password vanno salvate come hash: è una singola chiamata a `generate_password_hash`, funzione già inclusa in Flask, e non aggiunge nessuna complessità gestionale. La differenza pratica è che, se un domani qualcuno accede al file del database o a un backup, non si ritrova in mano l'elenco delle password del personale.

6. Tabella di marcia

Fase 0 — Validazione dell'interfaccia

Completata

Prototipo HTML autonomo, senza database, fatto provare al personale per confermare che la struttura della pagina e il flusso di lavoro corrispondano a come si lavora davvero in reparto. Serve a scoprire gli equivoci prima di scrivere il codice vero, non dopo.

Fase 1 — Applicazione funzionante in locale

Obiettivo: applicazione completa su un solo PC

1. Predisposizione dell'ambiente Python e della struttura del progetto.
2. Definizione dei modelli SQLAlchemy e script di creazione del database con i tre reparti e l'utenza amministratore.
3. Login e gestione della sessione.
4. Pagina Agenda pazienti in sola lettura, con la griglia dei giorni generata dalle date.
5. Inserimento ed eliminazione dei pazienti, modifica della gravità.
6. Salvataggio delle note giornaliere.
7. Note cliniche medico e infermiere con il controllo dei permessi.
8. Pagina di gestione utenti.
9. Esportazione CSV e registrazione nell'audit log.

Fase 2 — Prova sul campo

Almeno due settimane, con dati di fantasia

Far usare l'applicazione a un medico e a un infermiere su un PC di reparto, con nomi inventati. In questa fase emergono le cose che nessuna specifica scritta prevede: campi che mancano, campi che non serve nessuno, note troppo corte, ordinamenti scomodi. Le correzioni fatte qui costano molto meno di quelle fatte dopo il rilascio.

Fase 3 — Pubblicazione sulla VM

Da avviare solo dopo il via libera sul trattamento dei dati

Preparazione della macchina virtuale, PostgreSQL, Gunicorn come servizio, Nginx, restrizione dell'accesso alla sola LAN, backup automatico. Prima con dati di prova, poi con i dati reali.

Fase 4 — Consolidamento

Nei mesi successivi

- HTTPS con certificato emesso dalla CA interna del dominio.
- Verifica periodica del ripristino dei backup: un backup che non è mai stato ripristinato non è un backup.
- Eventuale autenticazione integrata con Active Directory, così il personale usa le credenziali di dominio e non una password in più.
- Archiviazione dei pazienti dimessi e storico consultabile.

7. Ambiente di sviluppo

L'editor è **Visual Studio Code**, non Visual Studio 2022. Sono due prodotti diversi con nomi simili: Visual Studio 2022 è l'ambiente completo orientato a C# e .NET, pesante e pensato per altri scenari; Visual Studio Code è un editor leggero ed è quello con cui si lavora normalmente in Python. Per questo progetto serve il secondo.

7.1 Cosa installare, nell'ordine

Passo 1 — Python 3.12

- Scaricare l'installer da `python.org/downloads`, versione 3.12 per Windows a 64 bit.
- Nella prima schermata dell'installer spuntare **"Add python.exe to PATH"** prima di procedere. È l'errore più comune: senza questa spunta i comandi non vengono riconosciuti dal terminale e reinstallare è l'unico modo semplice per rimediare.
- Scegliere "Install Now".
- Verifica: aprire PowerShell e digitare `py -3.12 --version`. Deve rispondere con il numero di versione.

Passo 2 — Visual Studio Code

- Scaricare da `code.visualstudio.com` la versione per Windows (User Installer, x64).
- Durante l'installazione spuntare le voci "Aggiungi l'azione Apri con Code al menu contestuale dei file" e "Aggiungi al PATH". Le prime due fanno comparire il comando nel tasto destro di Esplora risorse, la terza permette di aprire una cartella dal terminale con `code`.

Passo 3 — Git per Windows (consigliato)

- Scaricare da `git-scm.com` e installare accettando le opzioni predefinite.
- Non serve un account su servizi esterni: il repository resta in locale. Serve a poter tornare indietro quando una modifica rompe qualcosa, che nei primi mesi capita spesso.

7.2 Estensioni di Visual Studio Code

Si installano dall'icona dei quadrati nella barra laterale sinistra, oppure con `Ctrl+Shift+X`, cercando l'identificativo esatto indicato qui sotto.

- **Python** — `ms-python.python`. Fondamentale: esecuzione, debug, selezione dell'interprete.
- **Pylance** — `ms-python.vscode-pylance`. Si installa automaticamente insieme alla precedente. Fornisce il completamento e la segnalazione degli errori mentre scrivi.
- **Jinja** — `wholroyd.jinja`. Colora correttamente la sintassi dei template. Senza, i costrutti Jinja nei file HTML risultano illeggibili.

- **SQLite Viewer** — `qwtel.sqlite-viewer`. Permette di aprire `degenze.db` con un doppio clic e vedere il contenuto delle tabelle senza installare altri programmi. In fase di sviluppo è lo strumento che userai di più per capire cosa sta realmente succedendo nel database.
- **Ruff** — `charliermarsh.ruff`. Facoltativa. Segnala import inutilizzati, variabili non usate e formattazione incoerente.

7.3 Preparazione del progetto in Visual Studio Code

1. Creare la cartella `C:\cruscotto` con Esplora risorse.
2. In Visual Studio Code: menu **File** › **Apri cartella** e selezionare `C:\cruscotto`. Alla richiesta di fiducia sugli autori, confermare.
3. Creare l'ambiente virtuale: `Ctrl+Shift+P`, digitare *Python: Create Environment*, scegliere *Venv* e poi Python 3.12. L'editor crea la cartella `.venv` e la seleziona da solo.
4. Verificare che l'interprete sia quello giusto: in basso a destra deve comparire una dicitura del tipo *Python 3.12.x (.venv)*. Se non compare, `Ctrl+Shift+P` e *Python: Select Interpreter*.
5. Aprire il terminale integrato dal menu **Terminale** › **Nuovo terminale**. Se l'ambiente è attivo, il prompt inizia con `(.venv)`.
6. Installare i pacchetti: `pip install flask flask-sqlalchemy flask-login`

Errore che incontrerai al primo avvio del terminale

Se PowerShell risponde che l'esecuzione di script è disabilitata nel sistema, l'attivazione dell'ambiente virtuale non va a buon fine. Si risolve una volta sola, eseguendo `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` e confermando con S. La modifica vale solo per l'utente corrente e non tocca le impostazioni della macchina.

7.4 Configurazione del progetto

Due file dentro la sottocartella `.vscode` del progetto. Non sono obbligatori, ma il primo consente di avviare l'applicazione con il tasto F5 e di fermare l'esecuzione su un punto di interruzione per ispezionare le variabili, che è il modo più veloce per capire perché qualcosa non funziona.

`.vscode\launch.json`

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Cruscotto Degenze",
      "type": "debugpy",
      "request": "launch",
      "module": "flask",
      "env": { "FLASK_APP": "app.py", "FLASK_DEBUG": "1" },
      "args": ["run", "--host", "127.0.0.1", "--port", "5000"],
      "jinja": true
    }
  ]
}
```

.vscode\settings.json

```
{
  "python.defaultInterpreterPath": ".venv\\Scripts\\python.exe",
  "python.terminal.activateEnvironment": true,
  "files.associations": { "**/templates/*.html": "jinja-html" },
  "files.eol": "\r\n"
}
```

.gitignore

Da creare nella radice del progetto, se si usa Git. Serve a non mettere sotto controllo di versione né l'ambiente virtuale né il database, che contiene dati.

```
.venv/
__pycache__/
*.db
*.db-journal
.env
```

7.5 Comandi che userai ogni giorno

- **F5** — avvia l'applicazione in modalità debug. **Maiusc+F5** la ferma.
- **Ctrl+Shift+P** — apre la palette dei comandi, da cui si raggiunge tutto.
- **Ctrl+P** — apre rapidamente un file digitandone il nome.
- **Ctrl+ò** oppure menu Terminale — apre e chiude il terminale integrato.
- Clic sul margine sinistro accanto a un numero di riga — inserisce un punto di interruzione.

8. Installazione locale

Se hai già seguito il capitolo precedente, i primi due comandi sono superflui: Visual Studio Code ha già creato l'ambiente virtuale. Li riporto per completezza, nel caso tu debba rifare l'installazione da terminale.

Preparazione dell'ambiente

```
mkdir C:\cruscotto
cd C:\cruscotto
py -3.12 -m venv .venv
.venv\Scripts\Activate.ps1
pip install flask flask-sqlalchemy flask-login
```

Struttura del progetto

```
C:\cruscotto\  
  .venv\  
  app.py          avvio e rotte  
  modelli.py      tabelle SQLAlchemy  
  permessi.py     matrice dei ruoli  
  inicializza_db.py creazione database e dati iniziali  
  degenze.db      il database  
  templates\  
    base.html  
    login.html  
    agenda.html  
    utenti.html  
  static\  
    stile.css  
    agenda.js
```

Creazione del database e avvio

```
python inicializza_db.py  
flask --app app run --host 127.0.0.1 --port 5000
```

L'applicazione si apre su `http://127.0.0.1:5000`. L'opzione `--host 127.0.0.1` è quella che la rende invisibile alla rete: nessun altro PC può collegarsi e Windows non chiede di sbloccare nulla sul firewall.

Backup in questa fase

Chiudere l'applicazione e copiare il file `degenze.db`. Va fatto ogni volta prima di modifiche importanti al codice.

9. Messa in rete sulla macchina virtuale

I comandi sono per Ubuntu Server 24.04 LTS. Vanno eseguiti nell'ordine indicato, verificando a ogni passo prima di procedere.

Passo 1 — Preparazione della VM

- Macchina virtuale con 2 vCPU, 4 GB di RAM e 40 GB di disco: per venti utenti è abbondante.
- Indirizzo IP statico assegnato dalla riserva del datacenter.
- Record DNS di tipo A nella zona del dominio interno, per raggiungere l'applicazione con un nome anziché con l'indirizzo.
- Aggiornamento del sistema e installazione dei pacchetti: `sudo apt update && sudo apt install -y python3-venv postgresql nginx`

Passo 2 — Database

```
sudo -u postgres psql
CREATE DATABASE degenze;
CREATE USER cruscotto WITH PASSWORD 'scelta_dall_amministratore';
GRANT ALL PRIVILEGES ON DATABASE degenze TO cruscotto;
\q
```

Nel codice, la stringa di connessione passa da `sqlite:///degenze.db` a `postgresql+psycopg://cruscotto:...@localhost/degenze` e va letta da una variabile d'ambiente, non scritta nel file sorgente.

Passo 3 — Applicazione e Gunicorn

```
sudo mkdir -p /opt/cruscotto
cd /opt/cruscotto
python3 -m venv .venv
.venv/bin/pip install flask flask-sqlalchemy flask-login gunicorn psycopg[binary]
.venv/bin/python inizializza_db.py
.venv/bin/gunicorn -w 3 -b 127.0.0.1:8000 app:app
```

Verificare che risponda con `curl http://127.0.0.1:8000`, poi interrompere e trasformarlo in un servizio systemd, così riparte da solo al riavvio della macchina.

Passo 4 — Nginx

Nginx riceve sulla porta 80 e inoltra a `127.0.0.1:8000`. Gunicorn resta in ascolto solo sulla loopback e non è raggiungibile direttamente dalla rete.

Passo 5 — Restrizione dell'accesso

```
sudo ufw allow from 10.0.0.0/8 to any port 80
sudo ufw allow from 10.0.0.0/8 to any port 22
sudo ufw default deny incoming
sudo ufw enable
```

Le sottoreti vanno sostituite con quelle effettive della struttura. La regola predefinita di rifiuto garantisce che qualsiasi origine non esplicitamente consentita venga bloccata.

Passo 6 — Backup

Attività pianificata giornaliera con `pg_dump`, destinazione su un volume diverso da quello del database e conservazione degli ultimi trenta giorni. Va provato almeno un ripristino completo su una macchina di prova prima di considerare il sistema in produzione.

10. Decisioni ancora aperte

Questi punti vanno chiariti con il personale di reparto prima della Fase 1, perché incidono sul modello dati e cambiarli dopo costa molto di più.

- **Superamento del limite di giorni.** Cosa succede quando un paziente resta oltre i dieci o venti giorni previsti? La griglia si ferma, si allunga automaticamente, oppure il ricovero va rinnovato come nuovo ciclo?
- **Trasferimenti fra reparti.** Un paziente può passare da AOUP a un Setting? In caso affermativo conviene separare l'anagrafica del paziente dal singolo ricovero, con una tabella dedicata, per non perdere la storia del passaggio.
- **Sovrascrittura delle note giornaliere.** Attualmente la nota del giorno viene sostituita da chi scrive per ultimo. Se in reparto serve la tracciabilità di ogni intervento, anche le note giornaliere devono diventare voci in sola aggiunta come quelle cliniche.
- **Pazienti usciti.** Eliminarli cancella anche tutte le note collegate. Va deciso se serve conservare lo storico dei ricoveri conclusi e per quanto tempo.

Trattamento dei dati sanitari

L'applicazione tratta dati relativi alla salute di persone identificabili, che il GDPR classifica come categoria particolare. Prima del passaggio ai dati reali, anche restando sulla sola rete interna, va coinvolto il Responsabile della protezione dei dati della struttura per la valutazione del trattamento e l'inserimento nel registro. Fino ad allora, prototipo e prove vanno fatti esclusivamente con nomi di fantasia. È un passaggio amministrativo da avviare in anticipo, perché i tempi non dipendono dallo sviluppo.